

Claude Multi-Account Fine Tuning

Provider Aliases — User Guide

1. Prerequisites
2. Quick start
3. Commands
 - Useful flags
4. How a provider alias is named and what it runs
 - 4.1 Model tier — strongest paid if you have credit, strongest free if you don't
5. Config dirs, plugins, and shared state
 - Cross-alias session visibility (v1.5.0+)
 - Multi-alias system (v1.6.0+)
6. Overrides — `scripts/providers/overrides.json`
`scripts/providers/key-aliases.json`
7. Verification — what sync actually checks (v1.14.0+)
8. Known limitation — default session color
9. Troubleshooting
10. Remote distributed deployment & heavy testing
11. Command quick-reference
12. Individual provider notes
 - Kimi Code — OAuth subscription (kimi-for-coding, kimi-k3, kimi-k2p7, kimi-for-coding-highspeed)
 - z.ai Coding Plan (zai-coding-plan)
 - Xiaomi MiMo (xiaomi)
 - OpenCode Zen (opencode)
 - Chutes (chutes)
 - Poe (poe)
 - HelixAgent (helixagent) — local single-GPU model
13. Releasing — the mandatory live pre-release gate

Provider Aliases — User Guide

`claude-providers` turns every LLM API key you keep in your keys file into its own Claude Code alias, pointed at that provider's strongest model. It reuses the multi-account toolkit's machinery

(shared plugins/history, automatic `.bashrc/.zshrc` wiring) so a provider alias behaves like a normal `claudeN` account, just talking to a different model backend.

It is **fully dynamic**: the provider list, base URLs, transports, and model IDs are all derived at run time from your keys file + the [models.dev](#) catalog. Nothing is hardcoded. Re-run it any time to pick up new keys or refresh existing providers.

1. Prerequisites

Need	Why	Install
The toolkit installed	provides <code>lib.sh</code> , the alias file, <code>sync-state</code>	<code>bash scripts/install.sh</code>
A keys file	one export <code>NAME_API_KEY=...</code> per provider	<code>~/api_keys.sh</code> (default) or pass <code>--keys-file</code>
<code>jq</code> , <code>python3</code> , <code>curl</code>	catalog parse + resolve + fetch	system package manager
<code>claude-code-router</code> (only for routed providers)	translate Anthropic↔OpenAI/Gemini	<code>npm install -g @musistudio/claude-code-router</code>
submodules/ <code>LLMsVerifier</code> built (optional)	full "does this model work?" verification	see §7

Your keys file is the **only** place secrets live. `claude-providers` reads the variable *names* from it (without executing it) to discover providers, and the launch wrapper reads the *value* at the moment you start a session. No key is ever written into the repo, the alias file, or the per-provider env files.

2. Quick start

```
claude-providers sync          # discover + create/refresh all
                               provider aliases
source ~/.local/share/claude-multi-account/aliases.sh  # or open
                                                         a new shell
claude-providers list          # see what you got
deepseek                       # launch Claude Code on DeepSeek's
                               strongest model
```

3. Commands

Command	Does
<code>claude-providers sync</code> (<i>default</i>)	Discover every LLM key, resolve it via <code>models.dev</code> , verify it with live sentinel + tool-calling probes (see §7), and create/refresh one alias per provider. Idempotent — safe to run repeatedly.
<code>claude-providers list</code>	Table of verified provider aliases only (safe to launch): alias name, provider, transport, strong + fast model.
<code>claude-providers list-all</code>	Every installed provider alias, any status.
<code>claude-providers list-faulty</code>	Only aliases with an issue (failed / unverified / pending) — the filtered-out set.
<code>claude-providers show <id></code>	Print the resolved env file for one provider.
<code>claude-providers remove <id></code>	Remove the alias + env file; back up and remove the config dir.
<code>claude-providers add --from-key VAR --id PROVIDER</code>	Register a key→provider mapping (when a key's name doesn't match <code>models.dev</code>), then sync.

Useful flags

- `--keys-file PATH` — use a different keys file (default `~/api_keys.sh`).
- `--no-verify` — skip verification; create aliases regardless.
- `--offline` — don't fetch `models.dev`; use the local cache (errors if none).
- `--dry-run` — print what would change; write nothing.

4. How a provider alias is named and what it runs

- **Alias name** defaults to the `models.dev` provider id (deepseek, groq, mistral, ...). Want something shorter like dseek? Set it in `scripts/providers/overrides.json` (see §6).
- **Strong model** → Claude Code's main model (`ANTHROPIC_MODEL`). Chosen as the provider's most capable: reasoning-capable first, then newest, then largest context — **within the tier your account can actually pay for** (see §4.1).
- **Fast model** → Claude Code's background model (`ANTHROPIC_SMALL_FAST_MODEL`), chosen as the cheapest tool-call-capable model in that same tier.

- **Transport:**
 - native — provider speaks the Anthropic API directly (models.dev npm is @ai-sdk/anthropic). The alias runs claude with ANTHROPIC_BASE_URL / ANTHROPIC_AUTH_TOKEN / ANTHROPIC_MODEL set.
 - router — provider is OpenAI-compatible / Gemini. The alias launches through claude-code-router (ccr default-claude-code), which translates the protocol.

4.1 Model tier — strongest paid if you have credit, strongest free if you don't

Model selection is **credit-aware**. This is a hard rule applied to every provider alias:

Credit state of that provider's account	What the alias runs
Credit / purchased tokens available	the strongest paid model the provider serves that passes verification
No credit	the strongest free model (free tier / \$0 cost) that passes the same verification
Unknown — can't be determined	treated as <i>no credit</i> : the free choice

Why "unknown" falls back to free. The two possible mistakes cost very different amounts. Choosing a paid model on an unfunded key gives you a 402/403 at launch and a dead alias right when you wanted to work. Choosing a free model on a funded key only costs some capability, and the next claude-providers sync promotes it as soon as the credit signal becomes readable. So an inconclusive billing probe, a provider with no balance endpoint, or a stale offline catalog all resolve conservatively.

It's a floor on cost, not a cap on quality. Inside whichever tier applies, the pick is still the *strongest* model. The rule narrows the candidate set; it does not lower the bar within it. It applies to both the strong and the fast slot, and to every alias sync --multi generates.

Verification does not relax for free models. A free model still has to pass the sentinel probe and the tool-calling probe (\$7) before its alias is activated. "Cheapest that works" never degrades into "cheapest".

Your pin always wins. A strong_model / fast_model entry in scripts/providers/overrides.json (§6) overrides the automatic choice completely. Use it when you know exactly what you're paying for on a given provider and want the tier logic to stay out of it.

The enforcement mechanism (in `providers_resolve.py`, with matching detection in the bundled `LLMsVerifier`) landed alongside this section. The table and the rules above are the **behavioural contract**; check the source for the current flag and field names.

5. Config dirs, plugins, and shared state

Each provider gets `~/.claude-prov-<id>`, which symlinks the same shared items as your accounts (projects, todos, plugins, `settings.json`, `CLAUDE.md`, ...). So **every plugin you have in `claude1..N` is available in every provider session**, and history/projects are unified. `sync` also force-enables the always-on set in shared settings: **superpowers, systematic-debugging, frontend-design, code-review**.

Provider dirs are deliberately **excluded from account auto-detection**, so they never get merged into your real accounts' auth/identity and never interfere with `claude-unify` or `claude-add-account`. Your existing `claudeN` aliases keep working exactly as before, and you can still create new accounts the same way.

Cross-alias session visibility (v1.5.0+)

Sessions created under **any** alias — `claude1`, `claude2`, `deepseek`, `opencode`, `xiaomi`, or any other — are visible from **every** other alias via `/resume`. This works automatically:

1. **Before launch:** `claude-sync-state pull` merges every account's and provider's `.claude.json` into the launching dir. This includes `lastSessionId` (what `/resume` uses), `allowedTools`, MCP config, and all other project settings.
2. **After exit:** `claude-sync-state push` merges the post-session state back out, so the next alias to launch picks up the new session.
3. **Session files** are in `~/.claude-shared/sessions/`, which every alias symlinks to — so the session data itself was always shared; the new work ensures `.claude.json` project metadata is also merged.

Example workflow:

```
# Start a project as deepseek
cd /path/to/my-project
deepseek          # bare launch: auto-creates the project's
                  session
# ... work for a while, then exit
```

```
# Continue the same project as opencode
opencode          # bare launch: AUTO-RESUMES the same project
                  session – no /resume needed
```

A **bare** alias launch (no arguments) now auto-resumes — or first-time creates — the one long-lived session keyed to the project root, so switching aliases continues the same conversation automatically (see §12 of ../Claude_Multi_Account_Fine_Tuning.md and SESSION_COLOR.md). You only need /resume to switch to a **different** session than the project's own.

Performance: adds ~1-2 seconds per launch (jq merge across all dirs). Same overhead that claudeN aliases already have.

Note: provider dirs are still excluded from cma_detect_accounts (used by claude-unify and claude-add-account). Only claude-sync-state includes them, so the existing account management is unaffected.

Multi-alias system (v1.6.0+)

By default, sync creates **one alias per provider** using the 2 strongest models. With --multi, it verifies **ALL models** for each provider and creates **multiple aliases** — provider, provider2, provider3, etc. — each with a different pair of strong + fast models.

```
# Standard: 1 alias per provider (2 models)
claude-providers sync

# Multi: verify all models, create multiple aliases
claude-providers sync --multi

# Options
claude-providers sync --multi --max-aliases 10 --min-score 20
```

How verification works (v1.14.0+):

Each model is tested via live HTTP probes against the provider's chat endpoint: a sentinel probe (sends "Reply with exactly: VERIFY_OK" and **requires** the response to contain VERIFY_OK) and a tool-calling probe (requires the model to actually emit a tool call). Models are scored on 7 dimensions (0-100):

Dimension	Weight	What it checks
Existence + valid response	25pts	Model exists, returns real content
Tool calling	20pts	Supports function/tool_calls
Reasoning	15pts	

Dimension	Weight	What it checks
		Has chain-of-thought / reasoning_content
Context window	15pts	≥8K tokens (log scale)
Streaming	10pts	SSE streaming support
Latency	10pts	Response under 2s (full) or 5s (half)
Free tier	5pts	\$0 input cost

Tool calling is a hard gate, not just score. A model that answers chat but never calls tools is marked **not verified**, no matter how high its score — Claude Code is entirely tool-driven, so a tool-less model always breaks at runtime.

Anti-bluff detection prevents false positives:

- HTTP 200 with error body (JSON error wrapped in success)
- Empty response content
- **200 responses that lack the VERIFY_OK sentinel** (proxy fallback, silent model swap, canned text)
- Boilerplate "I'm unable to" responses
- Models that claim capability but don't deliver

Alias pairing:

- Models sorted by score, paired 2 per alias (strong + fast)
- Odd count: last model used for both positions in last alias
- Single model: used for both positions
- Default max: 5 aliases per provider (configurable)

Verification cache: results cached for 24h to avoid re-testing on every sync. Cache stored in `~/.local/share/claude-multi-account/providers/verification_cache.json`. The cache carries a schema version; results written by older verification logic (e.g. before tool calling was required) are ignored automatically.

6. Overrides — scripts/providers/overrides.json

Per-provider pins. Empty by default. Any field you set wins over the models.dev-derived value:

```
{
  "deepseek": {
    "alias": "dseek"
  }
}
```

This is how you (for example) give DeepSeek the short alias dseek — without touching any code.

A `strong_model` / `fast_model` pin here also **overrides the credit-aware tier choice** described in §4.1. If you pin a paid model on a provider whose credit state the toolkit reads as empty or unknown, that pin is honoured — the tier logic never second-guesses an explicit operator decision, so the launch failure that follows an unfunded key is then yours to expect.

Do not copy transport/base_url from an example. As of **v1.19.0** every provider — DeepSeek and Xiaomi included — is pinned to router transport on its OpenAI-compatible endpoint, because both were verified working there and a single uniform path is far easier to debug. The shipped pins are:

```
"deepseek": { "transport": "router", "base_url": "https://api.deepseek.com" }
"xiaomi":    { "transport": "router", "base_url": "https://api.xiaomimimo.com/v1" }
```

Pasting an older `"transport": "native" / .../anthropic` block would revert that fix. Always check the live values in `scripts/providers/overrides.json` before overriding transport or base_url.

scripts/providers/key-aliases.json

Maps a key variable name to a models.dev provider id, for the cases where the name differs (e.g. CODESTRAL_API_KEY → mistral, ZAI_API_KEY → zai). Use `claude-providers add --from-key VAR --id PROVIDER` to append entries.

7. Verification — what sync actually checks (v1.14.0+)

By default sync verifies every provider with **two live probes** against its chat endpoint, using the exact model the alias will run:

1. **Sentinel probe** — asks the model to reply with exactly `VERIFY_OK` and requires that token in the response. A 200 without it (or with an error object smuggled into the body) is a bluff: the endpoint answered *something*, not the requested model.
2. **Tool-calling probe** — requires the model to emit a real tool call (`tool_calls` / `tool_use`). Claude Code is tool-driven; a model that only chats is useless in practice.

Verdicts:

- **verified** — both probes passed. The alias is created and launchable.
- **failed** — a definitive problem: auth/billing rejection (401/402/403), model missing (404), missing sentinel, error-in-200, or no tool call. The alias is **not activated** (and the launch gate refuses to run it).
- **unverified** — the probes were inconclusive (rate-limit 429, 5xx, timeout, no network, no key). The alias is still created but the **launch gate** refuses to start it until a later sync reaches a verdict (override with `--force` — not recommended).

This replaced the old best-effort check (a bare `GET /models`), which proved only that the key was accepted — not that the model can actually run Claude Code. Both Anthropic-native (`/v1/messages`) and OpenAI-compatible (`/chat/completions`) endpoint shapes are probed in their native format.

For an additional authoritative layer ("can this model genuinely see and describe my code?") build the `LLMsVerifier` submodule:

```
git submodule update --init --depth 1 submodules/LLMsVerifier
cd submodules/LLMsVerifier && go build -o bin/model-
verification ./llm-verifier/cmd/model-verification
```

Once `submodules/LLMsVerifier/bin/model-verification` exists, `sync` prefers it as the existence check. `sync` also runs the two-round semantic code-visibility check (sentinel + independent judge) when it is available; billing/auth rejections there demote the alias, while transient judge/infra errors are an honest `SKIP` that never demotes. (Requires the Go toolchain; the build reads keys from your keys file.)

The live-TUI layer (`claude-providers verify <id> --deep`, `runner verify superpowers_tui.sh`) additionally **proves which backend served the turn**. Router-transport aliases share one `Router.default`, and an alias whose `base_url` is the gateway itself skips its own rewrite and would otherwise inherit the previous provider's route — a `PASS` that says nothing about the alias under test. Every evidence file now records `# ROUTE-INTENDED:` and `# ROUTE-RESOLVED:` (the latter read after the launch), and the leg fails with `# FAIL: route-mismatch` when they differ, or `# FAIL: route-unknown` when the route is unreadable. Native-transport aliases record an explicit `n/a` and are not route-checked.

Two further refusals close the remaining holes. `.Router.background` is checked alongside `.Router.default`, because Claude Code dispatches background sub-requests of the same turn through it — a mismatch there is `# FAIL: route-mismatch-background`. And because reading the config file back only proves what it says, the leg also demands a **restart receipt** bracketing the launch (a fresh gateway listening on line in `~/claude-code-router/service.log`, or a

changed service.json): the wrapper's ccr restart runs under `|| true` and can legitimately be refused, which would leave the previous provider serving while the file reads back correct. Without a receipt the leg fails closed with `# FAIL: route-unproven`. For router-transport aliases `jq` is a hard precondition — without it the route is unreadable and the leg refuses rather than skipping.

8. Known limitation — default session color

The original goal was for each provider session to default its /color to **purple**. Investigation of the installed Claude Code (2.1.195) found that /color is **session-scoped and TUI-only — it is never persisted to disk**, and there is no settings key, hook, or environment variable to set a default color. So this cannot be automated with the current Claude Code.

Workaround: type `/color purple` at the start of a provider session. `purple` is a valid color. If a future Claude Code adds a persistable color setting, `sync` will seed it automatically.

9. Troubleshooting

Symptom	Cause / fix
provider 'X' needs claude-code-router	Routed provider, ccr not installed → <code>npm install -g @musistudio/claude-code-router</code> .
<code>\$NAME_API_KEY</code> is empty	The key isn't set in your keys file.
offline and no <code>models.dev</code> cache	Run <code>claude-providers sync</code> once online to populate the cache.
A key didn't become an alias	It's unmapped (no <code>models.dev</code> match) or classified <code>vcs/infra</code> . Map it with <code>add</code> , or check <code>scripts/providers/evidence/mapping-report.md</code> .
Alias missing after sync	<code>source ~/.local/share/claude-multi-account/aliases.sh</code> or open a new shell.
400 "exceeded model token limit"	The catalog row for the model is self-inconsistent (<code>limit.output >= limit.context</code>). <code>derive_limits()</code> in <code>scripts/providers_resolve.py</code> always carves the output cap out of the context instead of trusting the published one, so such a row resolves to the same cap an absent output would; re-run <code>claude-providers sync</code> to rewrite the alias <code>.env</code> . A model the catalog has never heard of (an operator pin) no longer emits <i>no</i>

Symptom	Cause / fix
	limits either — an unknown context resolves to a conservative measured fallback (the catalog p10, narrowed by the provider's own p10, capped by that provider's widest window and floored so it stays usable), so both guards always exist. A :free row whose output budget exceeds its paid sibling's is only adjudicated at all when its output < context (where they are equal the row is just the common output-copied-from-context mislabel, which the carve already fixes), and its context is only lowered when at least three <i>distinct</i> providers other than the accused one , publishing the same model under a compatible vendor prefix, put it lower — then only to the lower median of what they publish. A lone suspicious row is left alone, because a wrongly-shrunk window is its own dead alias. Note the deliberate blind spot in the other direction: a genuinely throttled :free tier is corroborated at the <i>paid</i> value and left alone, since every peer record describes the paid tier.
# FAIL: route-mismatch in a --deep evidence file	The verification turn was served by a different backend than the alias under test (see §7). Re-run that leg on its own rather than after another router alias.
# FAIL: route-mismatch-background	Same, for the .Router.background entry: background sub-requests of that turn went to another backend (see §7).
# FAIL: route-unproven	The route was written but nothing proves the live gateway loaded it — the ccr restart was refused or never happened. Most often an authenticated gateway restarted without CCR_API_KEYS visible; re-run the leg with the gateway's keys in the environment.

10. Remote distributed deployment & heavy testing

For heavy testing that depends on **real production services**, the toolkit can boot the full LLMsVerifier System on a remote host (registered in config/containers/nezha.env) via the containers submodule + podman, and run real per-provider verification through it.

- **Deployment guide + boot procedure:** config/containers/llmsverifier/README.md (services, ports, the exact podman-compose boot steps, and every root-caused deployment fix).

- **What runs:** the llm-verifier API + an observability tier (prometheus + grafana with an auto-provisioned datasource & dashboard + node-exporter).
- **Heavy testing:** the bundled model-verification tool runs a real "Do you see my code?" check against each provider's live API:

```
ssh <user>@<host> 'podman run --rm --env-file ~/helix-system/
  llmsverifier/.env \
  llm-verifier-mv:nezha --provider deepseek --model deepseek-
  chat --verbose'
```

- **Evidence:** every verification + deployment claim is captured under docs/qa/20260616-infra/ (proofs, sweeps, security, observability).

11. Command quick-reference

```
# Provider aliases
claude-providers sync                # discover keys -> create/
  refresh aliases                    # skip verification (faster)
claude-providers sync --no-verify    # alias | provider |
claude-providers list                # transport | models
claude-providers show <id>           # one provider's resolved
  env                                # remove alias + env, back
claude-providers remove <id>         up config dir
claude-providers add --from-key VAR --id PROVIDER # register
  mapping + sync                    # non-interactive print mode

# Launch a provider session (after: source the alias file or open
  a new shell)
deepseek                             # router provider -> claude
  via ccr
<router-provider>                    # routed provider -> claude
  via ccr
deepseek -p "your prompt"            # non-interactive print mode

# Accounts (unchanged, still works)
claude-add-account --alias claudeN   # add a Claude account
claude-list-accounts                 # status of all accounts
claude-unify                         # re-merge shared state

# Docs
claude-export-docs                   # regenerate HTML/PDF/DOCX

# Release gate (run before tagging a release)
```

```
claude-release-gate # sandbox suite + LIVE real-  
alias smoke  
claude-release-gate --verify-providers # also run the full  
LLMsVerifier scan
```

12. Individual provider notes

Kimi Code — OAuth subscription (kimi-for-coding, kimi-k3, kimi-k2p7, kimi-for-coding-highspeed)

If you are signed into **Kimi Code** (the kimi CLI), every model your subscription serves becomes an alias automatically — **Kimi 3** (kimi-k3, 1M context, reasoning), **Kimi 2.7** (kimi-k2p7), kimi-for-coding-highspeed, and the account default kimi-for-coding. No API key is required; the OAuth session in ~/.kimi-code/credentials/kimi-code.json is used.

- **Discovery:** claude-providers sync queries GET /coding/v1/models with your OAuth token and emits one alias per served model (unioned with the models.dev catalog, because the listing under-reports). Each alias is verified with the strict sentinel + tool-calling + semantic pipeline.
- **Token freshness:** the OAuth token lives ~15 minutes. At every launch the wrapper reads the **live** credentials file (with expiry), refreshes via `kimi -p hi` when expired, and only then falls back to the sync-time snapshot — launches never die of a stale token.
- **kimi_proxy:** k3 enforces a "moonshot-flavored" JSON schema for tools (every \$ref must start with #/\$defs/). Claude Code's tool schemas would 400 without it, so all kimi-* launches route through a local normalizing proxy (scripts/proxy/kimi_proxy.py, installed by install.sh).
- **Precedence:** an OAuth subscription wins over KIMI_API_KEY / ApiKey_Kimi records for kimi-for-coding; the API keys remain the fallback on hosts without the OAuth session.

z.ai Coding Plan (zai-coding-plan)

The [z.ai](#) Coding Max-Yearly Plan provides access to Zhipu AI's flagship GLM models through a dedicated coding-optimized API endpoint. This is separate from the base z.ai plan — the Coding Plan endpoint uses `api.z.ai/api/coding/paas/v4` and includes **Free access** to glm-5.2 (the flagship with 1M context and reasoning) and glm-4.7 (the fast model with 204k context, reasoning, and tool_call support), among others.

How the alias works

The key variable `ZAI_API_KEY` in your keys file is mapped to the `zai-coding-plan` provider ID via `scripts/providers/key-aliases.json`. An override in `scripts/providers/overrides.json` pins the strong model to `glm-5.2` and the fast model to `glm-4.7`, which are the optimal choices for coding workloads on this plan.

Transport is **router** — the alias launches through `claude-code-router` (`ccr default-claude-code`), which translates the Anthropic protocol to the OpenAI-compatible `z.ai` API.

Models available on the Coding Plan

All models are free on the Coding Max-Yearly Plan:

Model	Context	Reasoning	Tool Call	Notes
glm-5.2	1M tokens	Yes	Yes	Flagship — newest, most capable
glm-5.1	—	Yes	Yes	Intermediate
glm-5-turbo	—	—	—	Turbo variant of glm-5
glm-5	—	—	—	Base generation 5
glm-4.7	204k tokens	Yes	Yes	Fast model — reasoning + tool_call
glm-4.6	—	—	—	Mid-range
glm-4.5-air	—	—	—	Lightweight
glm-4.5	—	—	—	Base model

Note on glm-5.2 rate limits: The Coding Max-Yearly Plan enforces a Fair Usage Policy on the flagship model: rapid-fire requests (e.g., repeated programmatic calls at short intervals) may trigger a temporary rate limit (error code 1313). Normal interactive use via `zai-coding-plan` at human pace is unaffected.

Setup

Everything is already configured on this host, but for a fresh install:

1. Ensure `ZAI_API_KEY` is exported in your keys file (`~/api_keys.sh`).
2. Run `claude-providers sync` to discover the key and create the alias.
3. `source ~/.local/share/claude-multi-account/aliases.sh` (or open a new shell).

4. Run `zai-coding-plan` to start a Claude Code session on `glm-5.2`.

No additional manual steps are needed — the key alias, model override, and endpoint are all defined in the provider config files.

Usage example

```
zai-coding-plan                                # launch Claude Code on glm-5.2
zai-coding-plan -p "explain this function"      # non-interactive
print mode
```

Xiaomi MiMo (xiaomi)

Xiaomi MiMo is Xiaomi's LLM platform. Unlike most OpenAI-compatible providers in this toolkit, MiMo exposes a genuine **Anthropic-native endpoint** (<https://api.xiaomimimo.com/anthropic>, `POST /anthropic/v1/messages`) that accepts `Authorization: Bearer <key>` and returns native Anthropic-format responses. Because of this, the `xiaomi` alias uses **native transport** — Claude Code talks to MiMo directly with no `claude-code-router` (`ccr`) dependency, the same way the `deepseek` alias works.

How the alias works

The key variable `XIAOMI_MIMO_API_KEY` in your keys file does **not** match the `models.dev` `xiaomi` provider's documented env name (`XIAOMI_API_KEY`), so `scripts/providers/key-aliases.json` maps `XIAOMI_MIMO_API_KEY` → `xiaomi`. An override in `scripts/providers/overrides.json` pins:

- **transport** `native`
- **base_url** `https://api.xiaomimimo.com/anthropic` (the Anthropic-native endpoint, overriding the catalog's OpenAI-compatible `/v1` URL)
- **strong_model** `mimo-v2.5-pro`
- **fast_model** `mimo-v2-flash`

The pinning is deliberate: `models.dev` lists a `mimo-v2.5-pro-ultraspeed` id that the **live API does not serve**, so the override guarantees only live-served model ids are used.

Models

MiMo's text-generation models (all support tool/function calling and reasoning; verified live 2026-06-19):

Model	Context	Reasoning	Tool Call	Notes
mimo-v2.5-pro	1M tokens	Yes	Yes	

Model	Context	Reasoning	Tool Call	Notes
				Flagship — strongest (alias default)
mimo-v2.5	1M tokens	Yes	Yes	Omni / multimodal (image, audio, video)
mimo-v2-pro	—	Yes	Yes	Legacy v2 Pro
mimo-v2-omni	256k tokens	Yes	Yes	Omni / multimodal (v2)
mimo-v2-flash	256k tokens	Yes	Yes	Fast model — cheapest tier (alias fast)

Non-text models exist (mimo-v2.5-asr, mimo-v2.5-tts*, mimo-v2-tts) but are speech-recognition / text-to-speech and cannot serve chat or code, so they are intentionally not wired as aliases.

Verified live

- GET /v1/models → HTTP 200, 10 models served (the 5 text models above + 5 ASR/TTS).
- POST /anthropic/v1/messages with Authorization: Bearer → HTTP 200, native Anthropic response (type:"message", content:[{type:"text"},{type:"thinking"}]) for both mimo-v2.5-pro and mimo-v2-flash.
- POST /v1/chat/completions with tools[] → finish_reason:"tool_calls", valid tool-call array, reasoning_content present (tool calling works).
- Streaming (stream:true) → SSE chat.completion.chunk deltas.
- Rate limits: 100 RPM / 10M TPM per account for text models (per-account, not per-key); expect 429 under load, retry with backoff.

Setup

Everything is already configured, but for a fresh install:

1. Ensure XIAOMI_MIMO_API_KEY is exported in your keys file (~/.api_keys.sh).
2. Run claude-providers sync to discover the key and create the alias.
3. source ~/.local/share/claude-multi-account/aliases.sh (or open a new shell).
4. Run xiaomi to start a Claude Code session on mimo-v2.5-pro.

Usage example

```
xiaomi                                # launch Claude Code on mimo-  
v2.5-pro  
xiaomi -p "explain this function"    # non-interactive print mode
```

OpenCode Zen (opencode)

OpenCode Zen is OpenCode's curated AI gateway — a tested and verified list of models from multiple providers, accessed through a single API key. Zen includes **21 free models** (all \$0 cost, all support tool calling and reasoning), plus 49 paid models from OpenAI, Anthropic, Google, DeepSeek, and others.

The free models are available for a limited time while OpenCode collects feedback. One of them — **Big Pickle** — is a stealth model (its true identity is not disclosed; observed as deepseek-v4-flash behind the scenes).

How the alias works

The key variable `ZEN_API_KEY` (or `ApiKey_Opencode_Zen`) in your keys file is mapped to the opencode provider ID via `scripts/providers/key-aliases.json`. An override in `scripts/providers/overrides.json` pins:

- **strong_model** big-pickle — free stealth model, 200K context, reasoning + tool_call
- **fast_model** deepseek-v4-flash-free — free, 200K context, reasoning + tool_call

Transport is **router** — the alias launches through `claude-code-router` (ccr default-`claude-code`), which translates the Anthropic protocol to the OpenAI-compatible Zen API (<https://opencode.ai/zen/v1/chat/completions>).

No transport or `base_url` override is needed — the `models.dev` catalog already has the correct values (`@ai-sdk/openai-compatible` → `router`, <https://opencode.ai/zen/v1>).

Free models available on Zen

All free models support tool calling and reasoning (verified live 2026-06-20):

Model	Context	Notes
big-pickle	200K	Stealth model — alias default (strong)
deepseek-v4-flash-free	200K	Alias fast model

Model	Context	Notes
mimo-v2.5-free	200K	Xiaomi MiMo
mimo-v2-pro-free	1M	Xiaomi MiMo Pro
mimo-v2-flash-free	262K	Xiaomi MiMo Flash
mimo-v2-omni-free	262K	Xiaomi MiMo Omni
nemotron-3-ultra-free	1M	NVIDIA Nemotron
nemotron-3-super-free	204K	NVIDIA Nemotron Super
north-mini-code-free	256K	North Mini Code
glm-4.7-free	204K	Zhipu GLM
glm-5-free	204K	Zhipu GLM 5
grok-code	256K	xAI Grok Code
kimi-k2.5-free	262K	Moonshot Kimi
minimax-m2.1-free	204K	MiniMax
minimax-m2.5-free	204K	MiniMax
minimax-m3-free	200K	MiniMax
qwen3.6-plus-free	262K	Alibaba Qwen
ring-2.6-1t-free	262K	Ring
hy3-preview-free	256K	HY3 Preview
ling-2.6-flash-free	262K	Ling (no reasoning)
trinity-large-preview-free	131K	Trinity (no reasoning)

Paid models start at \$0.05/1M tokens (GPT-5 Nano) and go up to \$30/1M (GPT-5.5 Pro). Claude models are available from \$1/1M (Haiku 4.5) to \$15/1M (Opus 4.1).

Verified live

- GET /v1/models → HTTP 200 (model list returned).
- POST /v1/chat/completions with big-pickle → HTTP 200, correct text response, cost=\$0, reasoning_content present. Stealth alias observed as deepseek-v4-flash.
- POST /v1/chat/completions with deepseek-v4-flash-free → HTTP 200, correct text, cost=\$0.
- Additional free models (mimo-v2.5-free, nemotron-3-ultra-free, north-mini-code-free) → all HTTP 200, all cost=\$0.
- Streaming (stream:true) → SSE chat.completion.chunk deltas.

Setup

Everything is already configured, but for a fresh install:

1. Ensure ZEN_API_KEY (or ApiKey_Opencode_Zen) is exported in your keys file (~/.api_keys.sh).

2. Run `claude-providers sync` to discover the key and create the alias.
3. `source ~/.local/share/claude-multi-account/aliases.sh` (or open a new shell).
4. Run `opencode` to start a Claude Code session on big-pickle (via ccr).

Usage example

```
opencode                                # launch Claude Code on big-
    pickle (via ccr)
opencode -p "explain this function" # non-interactive print mode
```

Notes

- The alias uses **router transport** (ccr) because Zen's free models use OpenAI-compatible format, not Anthropic native format.
- Big Pickle is a stealth model — the actual model served may vary. This is by design per OpenCode's documentation.
- To use paid Claude models on Zen instead of free models, override `strong_model` and `fast_model` in `scripts/providers/overrides.json` (e.g. `claude-haiku-4.5` and `claude-sonnet-4-6`), and set `transport` to `native` with `base_url` to `https://opencode.ai/zen/v1/messages`.

Chutes (chutes)

Chutes is a pay-per-use AI inference platform with **TEE (Trusted Execution Environment)** models — all models run in secure enclaves for data privacy. The API is OpenAI-compatible at <https://llm.chutes.ai/v1>.

How the alias works

The key variable `CHUTES_API_KEY` in your keys file maps to the chutes provider ID. An override in `scripts/providers/overrides.json` pins:

- **strong_model** `zai-org/GLM-5.2-TEE` — newest GLM model, 202K context
- **fast_model** `Qwen/Qwen3.6-27B-TEE` — fast Qwen model, 262K context

Transport is **router** — the alias launches through `claude-code-router` (ccr default-`claude-code`), which translates the Anthropic protocol to the OpenAI-compatible Chutes API.

Models available on Chutes

All models are TEE-enabled (Trusted Execution Environment):

Model	Context	Notes
zai-org/GLM-5.2-TEE	202K	Newest GLM — alias default (strong)
zai-org/GLM-5.1-TEE	202K	GLM 5.1
zai-org/GLM-5-TEE	202K	GLM 5
Qwen/Qwen3.5-397B-A17B-TEE	262K	Largest Qwen model
Qwen/Qwen3.6-27B-TEE	262K	Alias fast model
Qwen/Qwen3-32B-TEE	40K	Qwen 32B
Qwen/Qwen3-235B-A22B-Thinking-2507-TEE	262K	Thinking/reasoning model
deepseek-ai/DeepSeek-V3.2-TEE	131K	DeepSeek V3.2
moonshotai/Kimi-K2.5-TEE	262K	Kimi K2.5
moonshotai/Kimi-K2.6-TEE	262K	Kimi K2.6
MiniMaxAI/MiniMax-M2.5-TEE	196K	MiniMax M2.5
google/gemma-4-31B-turbo-TEE	131K	Google Gemma 4
unsloth/Mistral-Nemo-Instruct-2407-TEE	131K	Mistral Nemo

Important: Chutes is **pay-per-use** — the account must be funded to use models. Add balance at <https://chutes.ai>. The API key and configuration are correct, but requests will return "Quota exceeded" if the account balance is \$0.

Setup

1. Ensure CHUTES_API_KEY is exported in your keys file (~/.api_keys.sh).
2. Fund your Chutes account at <https://chutes.ai>.
3. Run `claude-providers sync` to discover the key and create the alias.
4. `source ~/.local/share/claude-multi-account/aliases.sh` (or open a new shell).
5. Run `chutes` to start a Claude Code session on `zai-org/GLM-5.2-TEE`.

Usage example

```
chutes                                # launch Claude Code on
    GLM-5.2-TEE (via ccr)
chutes -p "explain this function"    # non-interactive print mode
```

Verified

- API endpoint `https://llm.chutes.ai/v1/chat/completions` responds correctly
- Authentication with `Authorization: Bearer cpk_...` works
- All 13 TEE models are accessible (require funded account)
- OpenAI-compatible format confirmed

Poe (poe)

Poe is a universal AI platform with **382 models** from all major providers — chat, code, image generation, video generation, TTS, and STT. The API is OpenAI-compatible at `https://api.poe.com/v1`.

How the alias works

The key variable `POE_API_KEY` (or `ApiKey_Poe`) in your keys file maps to the poe provider ID. Transport is **router** — the alias launches through `claude-code-router` (`ccr default-claude-code`).

Aliases

Alias	Strong Model	Fast Model	Focus
poe	claude-sonnet-4.6	gpt-5.4-mini	Primary — best Claude
poe2	gpt-5.5	deepseek-v4-pro-e	GPT-focused
poe3	grok-4	gemini-3.1-pro	Alternative providers

Model categories (382 total)

Category	Count	Examples
Chat/Reasoning	130	claude-opus-4.8, gpt-5.5, grok-4, deepseek-v4-pro, gemini-3.1-pro, qwen3.7-max
Code	16	claude-code, gpt-5.3-codex, qwen3-coder-next, kimi-k2.7-code, seed-2.0-code
	40	

Category	Count	Examples
Image Generation		flux-2-pro, imagen-4, stable-diffusion3.5, qwen-image-2, grok-image-image
Video Generation	17	sora-2-pro, veo-3.1, kling-3.0, runway-gen-4.5, pixverse-v5.6
TTS/Voice	12	elevenlabs-v3, gemini-3.1-flash-tts, cartesia-ink-whisper, orpheus-tts
STT/Speech	1	whisper-v3-large-t
Other	166	perplexity-search, exa-research, hailuo, minimax-m3, glm-5.2, and more

Supported capabilities

- **Chat completions** — OpenAI-compatible `/v1/chat/completions`
- **Tool calling** — verified on claude-sonnet-4.6, gpt-5.4-mini, deepseek-v4-pro-e, grok-4
- **Streaming** — standard SSE streaming supported
- **Reasoning** — `reasoning_content` field in responses
- **Image input** — vision models support image URLs
- **Video generation** — via separate video endpoint
- **TTS/STT** — text-to-speech and speech-to-text models

Setup

1. Ensure `POE_API_KEY` is exported in your keys file (`~/api_keys.sh`).
2. Run `claude-providers sync` to discover the key and create the alias.
3. `source ~/.local/share/claude-multi-account/aliases.sh` (or open a new shell).
4. Run `poe` to start a Claude Code session on `claude-sonnet-4.6` (via `ccr`).

Usage example

```
poe                                # launch Claude Code on
                                claude-sonnet-4.6 (via ccr)
poe2                              # launch on gpt-5.5 (via ccr)
poe3                              # launch on grok-4 (via ccr)
poe -p "explain this function"    # non-interactive print mode
```

Verified

- API endpoint `https://api.poe.com/v1/chat/completions` responds correctly
- Authentication with `Authorization: Bearer sk-poe-...` works
- Tool calling verified on multiple models

- 382 models accessible across all categories
- OpenAI-compatible format confirmed

HelixAgent (helixagent) — local single-GPU model

helixagent points Claude Code at a **local** HelixLLM backend — a podman container serving Qwen3-Coder-30B on one GPU (an RTX 5090) at `http://127.0.0.1:18434/v1` — instead of a hosted API. Transport is **router**: launches go through `claude-code-router` plus the bundled Go `cma-proxy`, which recovers the model's Hermes/Qwen `<function=...>` tool calls into structured `tool_calls` so Claude Code's tools actually engage. The alias is pinned in `scripts/providers/helixagent.json` to a **229,376-token** context window and an **8,192-token** output cap.

Two things are specific to a local, single-GPU backend and worth knowing before you launch it.

The backend must be in "claude mode" (one big slot), not "coder mode"

The HelixLLM backend is shared — on the **same single GPU** — with HelixCode, and the two configurations are **mutually exclusive**:

Mode	llama.cpp config	Slot layout	For
claude mode	<code>-c 229376 --parallel 1</code>	one 229,376-token slot	the helixagent alias (Claude Code)
coder mode	<code>-c 24576 --parallel 8</code>	eight 3,072-token slots	HelixCode's concurrent sub-requests

llama.cpp splits `-c` across the parallel slots, so **coder mode gives each request only ~3,072 tokens** — a Claude Code session (system prompt + tool schemas) is far larger, and every launch against coder mode returns **HTTP 400**. helixagent needs the single large slot of claude mode.

Because coder mode is the **default** operational state (HelixCode is the common case), helixagent is honestly demoted to **unverified** and the launch gate refuses it until you flip the backend to claude mode:

```
# The mode-switch script lives in the companion helix_code repo —
  NOT this toolkit:
helix_code/scripts/helixllm-mode.sh claude
  # one 229,376-token slot
# then re-verify + launch:
```

```
claude-providers verify helixagent --deep          # passes once the
              backend is in claude mode
helixagent
```

Switch back with `helixllm-mode.sh coder` when you need HelixCode again. The two modes cannot run at once on the shared GPU.

Minimal-launch (--bare) trim mode — CMA_PROVIDER_TRIM='bare'

A local 229,376-token window is small next to a hosted model's, and a normal launch would overflow it before the first turn: on a plugin-heavy host, auto-resumed session history (~330k tokens) plus the fixed hook/plugin/MCP/CLAUDE.md tool schemas (~110k tokens) together dwarf the whole window. So helixagent's resolved env file carries `CMA_PROVIDER_TRIM='bare'`, which makes **every conversation launch minimal and fresh**:

- prepends **--bare**, dropping the hook/plugin/MCP/CLAUDE.md surface;
- **skips both automatic history seams** — the conversation-args auto---resume injection *and* the interactive (zero-args) stored session-flags injection — so no synced session history rides along;
- so each launch is a **fresh session that fits the local window**.

What trim deliberately does **not** touch:

- **Explicit** session selectors you pass yourself (`--resume <id>`, `--session-id <id>`, `--continue`, `--fork-session`, `-c`) are honored **verbatim** — trim only suppresses the *automatic* resume, never your own.
- **Non-conversation subcommands** (`agents`, `mcp`, `export`, `doctor`, `config`, `plugin`, `setup`, ...) get neither `--bare` nor a resume.
- **Untrimmed providers are byte-identical to before**. Trim is opt-in per provider via the `.env` line, wired today only for helixagent but reusable for any local-model provider that needs a fresh minimal session each launch.

13. Releasing — the mandatory live pre-release gate

`claude-release-gate` is the release gate: **no release commit may be made unless it exits 0**. It exists because a green sandbox suite is not proof the real host works — v1.25.1 shipped with the entire sandbox suite green while **every** router alias on the real host was bricked. The sandbox proves wrapper *logic*; it is structurally blind to real-host state (a `PATH`-shadowing `ccr`

doppelgänger, a mis-configured local backend, resumed history overflowing a small window). So the gate adds a **live** layer that drives the real chain end-to-end.

```
claude-release-gate          # sandbox suite + live
                             smoke (default provider: helixagent)
claude-release-gate --provider poe      # gate through a
                             different provider
claude-release-gate --skip-suite        # reuse a suite run you
                             JUST completed green
claude-release-gate --verify-providers  # also run the full
                             LLMsVerifier provider scan
```

The layers, fail-closed — any failure means **DO NOT RELEASE**:

1. **Sandbox suite** — scripts/tests/run-all.sh. --skip-suite skips it, only valid if you just ran it green.
2. **Live alias smoke** — regenerates the aliases from the *current* lib.sh, then launches the real alias with a fresh session (... --session-id <uuid> -p "Reply with exactly: GATE-OK") through the real **PATH** → **ccr** → **route-apply** → **proxy** → **provider backend**, and asserts (a) the launch exited 0, (b) the served reply contained GATE-OK, and (c) for a router-transport provider, that the gateway's **sink-side route** (.Router.default in ~/.claude-code-router/config.json) actually names the provider under test. A write-then-apply route that silently served the wrong backend fails here.
3. **Provider scan** (*opt-in*, --verify-providers) — runs claude-verify-providers (the LLMsVerifier scan) over every provider/model. Slower; off by default.

The gate provider is chosen as --provider, else \$CMA_GATE_PROVIDER, else helixagent. It **must exist and be verified** — a missing or broken gate provider is a gate **failure** (fix it, or pick another with --provider), never a silent skip. For helixagent specifically that means the HelixLLM backend must be in claude mode first (§12).